

## LAB ASSIGNMENT-8.3

2303A51343

BATCH-10

### **TASK-1:**

#### **PROMPT:**

write a python program to develop a user registration system that requires reliable email input validation.

#### **CODE:**

```
import re

class UserRegistration:

    def __init__(self, name, email):

        self.name = name

        self.email = email

    def validate_email(self):

        pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

        if re.match(pattern, self.email):

            return True

        else:

            return False

    def display(self):

        print(f"Name: {self.name}")

        print(f"Email: {self.email}")

        if self.validate_email():

            print("Email is valid.")

        else:

            print("Email is invalid.")

# Test Cases

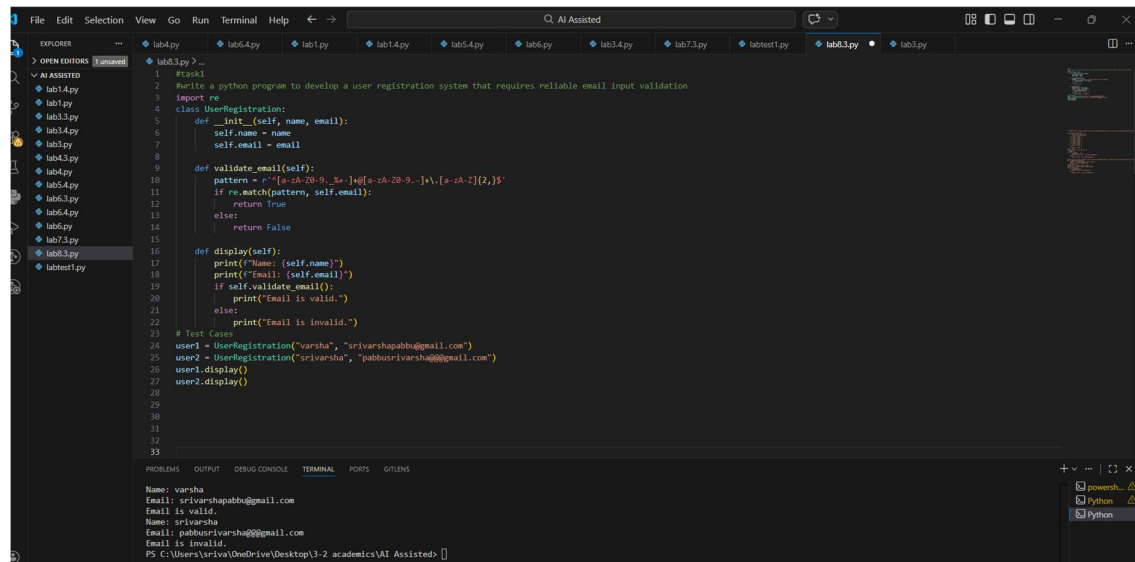
user1 = UserRegistration("varsha", "srivarshapabbu@gmail.com")
```

```
user2 = UserRegistration("srivarsha", "pabbusrivarsha@@@gmail.com")
```

```
user1.display()
```

```
user2.display()
```

## OUTPUT:



The screenshot shows a VS Code editor with a Python file named `lab6.3.py`. The code defines a `UserRegistration` class with methods `__init__`, `validate_email`, and `display`. It includes a regular expression for email validation and test cases. The terminal output shows the results of the test cases:

```
Name: srivarsha
Email: srivarshapabb@gmail.com
Email is valid.
Name: srivarsha
Email: pabbusrivarsha@@@gmail.com
Email is invalid.
PS C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted >
```

## Analysis:

It creates a user registration system that validates email format using a regular expression and prints whether the entered email is valid or invalid.

## TASK-2

### PROMPT:

write a python program to bulid an automated grading system for an online examination platform using loops and aslo include invalid input like -5,eighty.

### CODE:

```
def calculate_grade(score):
```

```
    if score < 0 or score > 100:
```

```
        return "Invalid input"
```

```
    elif score >= 90:
```

```
        return "Grade: A"
```

```
    elif score >= 80:
```

```
        return "Grade: B"
```

```
elif score >= 70:
```

```
    return "Grade: C"
```

```
elif score >= 60:
```

```
    return "Grade: D"
```

```
else:
```

```
    return "Grade: F"
```

```
# Test Cases
```

```
scores = [95, 85, 75, 65, 55, -5, "eighty"]
```

```
for s in scores:
```

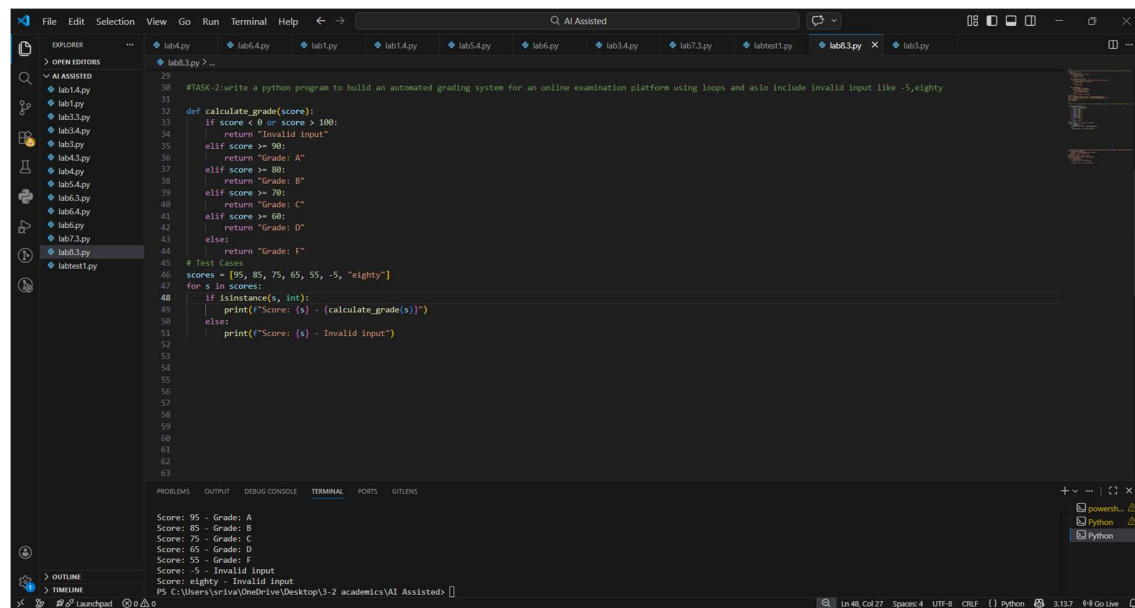
```
    if isinstance(s, int):
```

```
        print(f"Score: {s} - {calculate_grade(s)}")
```

```
    else:
```

```
        print(f"Score: {s} - Invalid input")
```

## OUTPUT:



The screenshot shows a VS Code editor with a Python script in the main editor and its output in the terminal. The script defines a function `calculate_grade(score)` that returns grades A through F based on score ranges. It then tests the function with a list of scores, including invalid inputs like -5 and "eighty". The terminal output shows the results of these tests.

```
20
21 #TASK-2:write a python program to build an automated grading system for an online examination platform using loops and also include invalid input like -5,eighty
22
23 def calculate_grade(score):
24     if score < 0 or score > 100:
25         return "Invalid input"
26     elif score >= 90:
27         return "Grade: A"
28     elif score >= 80:
29         return "Grade: B"
30     elif score >= 70:
31         return "Grade: C"
32     elif score >= 60:
33         return "Grade: D"
34     else:
35         return "Grade: F"
36
37 # Test Cases
38 scores = [95, 85, 75, 65, 55, -5, "eighty"]
39 for s in scores:
40     if isinstance(s, int):
41         print(f"Score: {s} - {calculate_grade(s)}")
42     else:
43         print(f"Score: {s} - Invalid input")
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
```

Score: 95 - Grade: A  
Score: 85 - Grade: B  
Score: 75 - Grade: C  
Score: 65 - Grade: D  
Score: 55 - Grade: F  
Score: -5 - Invalid input  
Score: eighty - Invalid input

## Analysis:

It assigns grades using conditions and a loop, and checks for invalid inputs like negative numbers or non-numeric values before grading.

### TASK-3:

#### PROMPT:

write a python program to check sentence palindrome checker to develop a text-processing utility to analyze sentences.

#### CODE:

```
def is_palindrome(sentence):

    cleaned = ".join(sentence.lower().split())

    return cleaned == cleaned[::-1]

# Test Cases

sentences = ["Madam In Eden Im Adam",

"Hello World","A man a plan a canal Panama"]

for s in sentences:

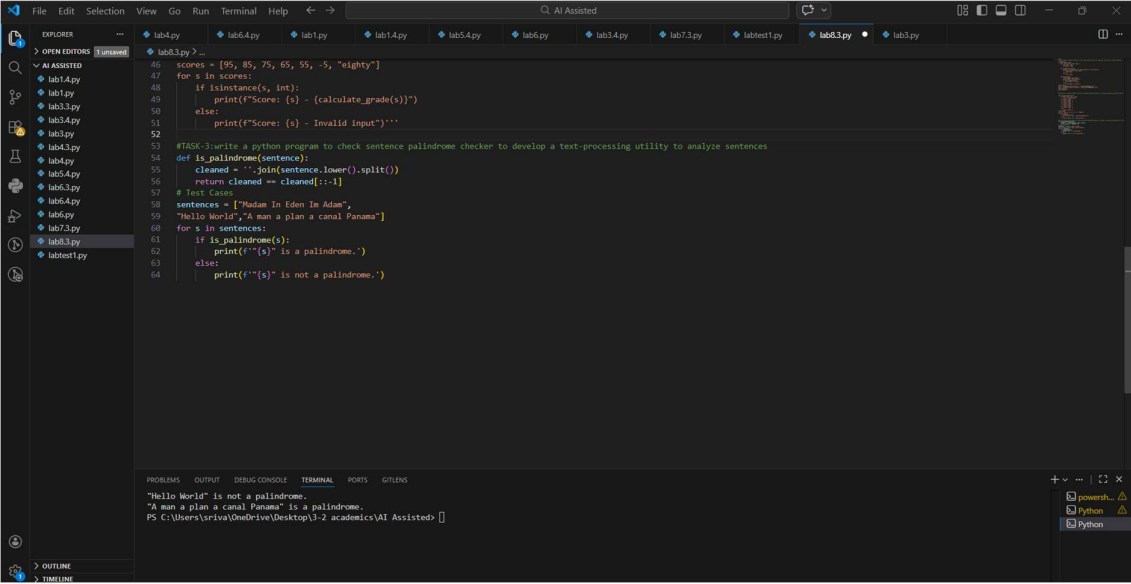
    if is_palindrome(s):

        print(f'"{s}" is a palindrome.')

    else:

        print(f'"{s}" is not a palindrome.')
```

#### OUTPUT:



```
File Edit Selection View Go Run Terminal Help
lab8.3py
46 scores = [99, 85, 75, 65, 55, -5, "eighty"]
47 for s in scores:
48     if isinstance(s, int):
49         print(f"Score: {s} - {calculate_grade(s)}")
50     else:
51         print(f"Score: {s} - Invalid input")
52
53 #TASK-3:write a python program to check sentence palindrome checker to develop a text-processing utility to analyze sentences
54 def is_palindrome(sentence):
55     cleaned = ".join(sentence.lower().split())
56     return cleaned == cleaned[::-1]
57
58 # Test Cases
59 sentences = ["Madam In Eden Im Adam",
60 "Hello World","A man a plan a canal Panama"]
61 for s in sentences:
62     if is_palindrome(s):
63         print(f'"{s}" is a palindrome.')
64     else:
65         print(f'"{s}" is not a palindrome.')
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GIT LENS

```
"Hello World" is not a palindrome.
"A man a plan a canal Panama" is a palindrome.
PS C:\Users\sriva\OneDrive\Desktop>3-2 academics\AI Assisted
```

**Analysis:**

It checks whether a sentence is a palindrome by converting it to lowercase, removing spaces, and comparing the string with its reverse.

**TASK-4:****PROMPT:**

Write a python program to design a basic shopping cart module for an e-commerce application.add item and remove item and total cost.

**CODE:**

```
class ShoppingCart:

    def __init__(self):

        self.cart = {}

    def add_item(self, item, price):

        self.cart[item] = price

        print(f"Added {item} to cart at ₹{price}")

    def remove_item(self, item):

        if item in self.cart:

            del self.cart[item]

            print(f"Removed {item} from cart")

        else:

            print(f"{item} not found in cart")

    def total_cost(self):

        return sum(self.cart.values())

# Test Cases

cart = ShoppingCart()

cart.add_item("Laptop", 50000)

cart.add_item("Headphones", 2000)

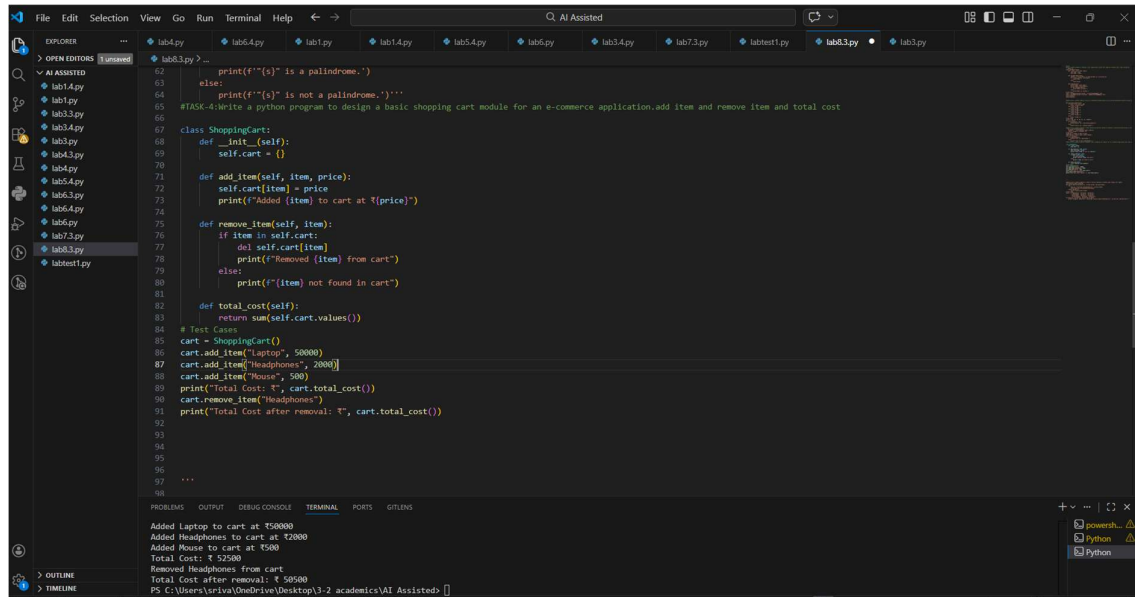
cart.add_item("Mouse", 500)

print("Total Cost: ₹", cart.total_cost())

cart.remove_item("Headphones")
```

```
print("Total Cost after removal: ₹", cart.total_cost())
```

## OUTPUT:



```
62     print(f"{s}" is a palindrome.')
63 else:
64     print(f"{s}" is not a palindrome.')'''
65 #TASK-4:Write a python program to design a basic shopping cart module for an e-commerce application.add item and remove item and total cost
66
67 class ShoppingCart:
68     def __init__(self):
69         self.cart = {}
70
71     def add_item(self, item, price):
72         self.cart[item] = price
73         print(f"Added {item} to cart at ₹{price}")
74
75     def remove_item(self, item):
76         if item in self.cart:
77             del self.cart[item]
78             print(f"Removed {item} from cart")
79         else:
80             print(f"{item} not found in cart")
81
82     def total_cost(self):
83         return sum(self.cart.values())
84
85 # Test Cases
86 cart = ShoppingCart()
87 cart.add_item("Laptop", 50000)
88 cart.add_item("Headphones", 2000)
89 cart.add_item("Mouse", 500)
90 print("Total Cost: ₹", cart.total_cost())
91 cart.remove_item("Headphones")
92 print("Total Cost after removal: ₹", cart.total_cost())
93
94
95
96
97 ...
98
```

Added Laptop to cart at ₹50000  
Added Headphones to cart at ₹2000  
Added Mouse to cart at ₹500  
Total Cost: ₹ 52500  
Removed Headphones from cart  
Total Cost after removal: ₹ 50500  
PS C:\Users\srivala\OneDrive\Desktop\3-2 academics\AI Asisted> |

## Analysis:

It creates a ShoppingCart class to add and remove items and calculate the total price using a dictionary, where item names are keys and prices are values.

## Task-5

### PROMPT:

write a python program to create a utility function to convert date formats for reports.

### CODE:

```
from datetime import datetime
```

```
def convert_date_format(date_str, current_format, desired_format):
```

```
    try:
```

```
        date_obj = datetime.strptime(date_str, current_format)
```

```
        return date_obj.strftime(desired_format)
```

```
    except ValueError:
```

```
        return "Invalid date format"
```

```
# Test Cases
```

```
dates = [("2024-06-15", "%Y-%m-%d", "%d/%m/%Y"),
```

```
("15/06/2024", "%d/%m/%Y", "%Y-%m-%d"),

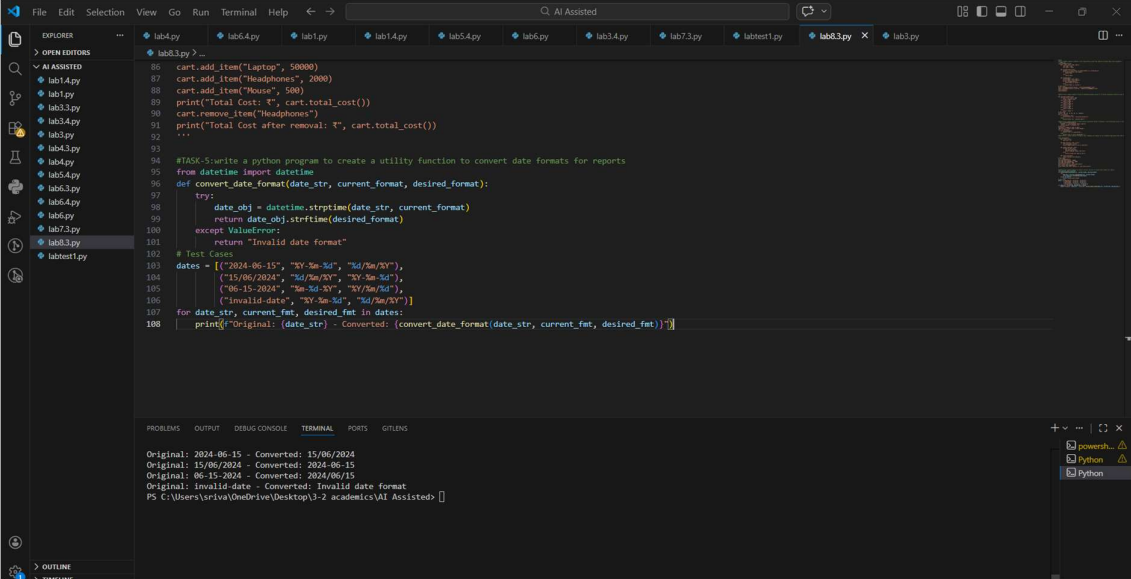
("06-15-2024", "%m-%d-%Y", "%Y/%m/%d"),

("invalid-date", "%Y-%m-%d", "%d/%m/%Y")]
```

for date\_str, current\_fmt, desired\_fmt in dates:

```
    print(f"Original: {date_str} - Converted: {convert_date_format(date_str, current_fmt,
desired_fmt)}")
```

## OUTPUT:



The screenshot shows a VS Code editor with a Python script in the main editor and its output in the terminal. The script defines a function `convert_date_format` that takes a date string, a current format, and a desired format, and returns the converted date string. It uses `datetime.strptime` to parse the date and `datetime.strftime` to format it. The script also includes a list of test cases and a loop that prints the original and converted dates for each case.

```
86 cart.add_item("Laptop", 50000)
87 cart.add_item("Headphones", 2000)
88 cart.add_item("House", 500)
89 print("Total Cost: ₹", cart.total_cost())
90 cart.remove_item("Headphones")
91 print("Total Cost after removal: ₹", cart.total_cost())
92 ...
93
94 #TASK-5:write a python program to create a utility function to convert date formats for reports
95 from datetime import datetime
96 def convert_date_format(date_str, current_format, desired_format):
97     try:
98         date_obj = datetime.strptime(date_str, current_format)
99         return date_obj.strftime(desired_format)
100     except ValueError:
101         return "Invalid date format"
102
103 # Test Cases
104 dates = [
105     ("2024-06-15", "%Y-%m-%d", "%d/%m/%Y"),
106     ("15/06/2024", "%d/%m/%Y", "%Y-%m-%d"),
107     ("06-15-2024", "%m-%d-%Y", "%Y/%m/%d"),
108     ("invalid-date", "%Y-%m-%d", "%d/%m/%Y")]
109
110 for date_str, current_fmt, desired_fmt in dates:
111     print(f"Original: {date_str} - Converted: {convert_date_format(date_str, current_fmt, desired_fmt)}")
```

The terminal output shows the results of the script execution:

```
Original: 2024-06-15 - Converted: 15/06/2024
Original: 15/06/2024 - Converted: 2024-06-15
Original: 06-15-2024 - Converted: 2024/06/15
Original: invalid-date - Converted: Invalid date format
PS C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted>
```

## Analysis:

This program converts a date from one format to another using `datetime`.

It reads the date with `strptime()`, changes the format with `strftime()`, and uses `try-except` to handle invalid dates.